

ARTIFICIAL NEURAL NETWORK (ANN)
MACHINE LEARNING MODEL FRAMEWORK
FOR ANALOG CIRCUIT DESIGN OPTIMISATION

GOH WEI CUN

UNIVERSITI TEKNOLOGI MALAYSIA



**UNIVERSITI TEKNOLOGI MALAYSIA
DECLARATION OF UNDERGRADUATE REPORT**

Author's full name : Goh Wei Cun
 Student's Matric No. : A22EE0278 Academic Session : 25/26 - 1
 Date of Birth : 27 March 2003 UTM Email : gohcun@graduate.utm.my
 Report Title : ARTIFICIAL NEURAL NETWORK (ANN)
 MACHINE LEARNING MODEL FRAMEWORK
 FOR ANALOG CIRCUIT DESIGN OPTIMISATION

I declare that this thesis is classified as:

☒

OPEN ACCESS

I agree that my report to be published as a hard copy or made available through online open access.

☐

RESTRICTED

Contains restricted information as specified by the organization/institution where research was done.
(The library will block access for up to three (3) years)

☐

CONFIDENTIAL

Contains confidential information as specified in the Official Secret Act 1972)

(If none of the options are selected, the first option will be chosen by default)

I acknowledged the intellectual property in the report belongs to Universiti Teknologi Malaysia, and I agree to allow this to be placed in the library under the following terms :

1. This is the property of Universiti Teknologi Malaysia
2. The Library of Universiti Teknologi Malaysia has the right to make copies for the purpose of only.
3. The Library of Universiti Teknologi Malaysia is allowed to make copies of this report for academic exchange.

Signature of Student:

Signature :

Full Name: GOH WEI CUN

Date : 16 JANUARY 2026

Approved by Supervisor(s)

Signature of Supervisor I:

Full Name of Supervisor I

MOHD AZHAR BIN ABDUL RAZAK

Date :

NOTES : If the thesis is CONFIDENTIAL or RESTRICTED, please attach with the letter from the organization with period and reasons for confidentiality or restriction

“I hereby declare that I have read this report and in my
opinion this report is sufficient in term of scope and quality for the
award of the degree of Bachelor of Electronic Engineering with Honours”

Signature : _____
Name of Supervisor : MOHD AZHAR BIN ABDUL RAZAK
Date : 16 JANUARY 2026

ARTIFICIAL NEURAL NETWORK (ANN)
MACHINE LEARNING MODEL FRAMEWORK
FOR ANALOGUE CIRCUIT DESIGN OPTIMISATION

GOH WEI CUN

A report submitted in partial fulfilment of the
requirements for the award of the degree of
Bachelor of Electronic Engineering with Honours

Faculty of Electrical Engineering
Universiti Teknologi Malaysia

JANUARY 2026

DECLARATION

I declare that this report entitled “*Artificial Neural Network (ANN) Machine Learning Model Framework for Analogue Circuit Design Optimisation*” is the result of my own research except as cited in the references. The report has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Signature :
Name : GOH WEI CUN
Date : 16 JANUARY 2026

ACKNOWLEDGEMENT

When going through my Final Year Project (FYP), a lot of lecturers have given me solid advice and suggestions on ways to implement my project. Among them include my supervisor, Dr Mohd Azhar Abdul Razak, who is constantly overseeing my progress and providing constructive input. There is also Dr Jamalauddin Zakaria, who has given sound comments on my chosen title.

I am also grateful to Univerisiti Teknologi Malaysia (UTM) for providing the relevant tools for the implementation of my FYP, not limited to software licenses, but resources available campus-wide.

ABSTRACT

Currently, designing analogue circuits still remains a repetitive and manual task that relies on human's expertise for trade-offs to achieve the optimal performance. As such, the purpose of this project is to create an Electronic Design Automation (EDA) tool for optimising analogue circuit designs, specifically operational amplifiers (op-amps) circuits. This will be achieved via the use of an Artificial Neural Network (ANN) machine learning framework that will predict all transistor devices' sizing found in the circuit. The ANN will be trained on datasets produced via simulation in Cadence Virtuoso with OCEAN scripting. In development, two models of the ANN will be produced: a performance model (feed-forward) and an optimization model (backwards propagation). Cascading multi-stage op-amps will be the focus of this project, which includes the three-stage Reversed Nested Miller Compensation (RNMC) op-amp, to stay relevant with current industrial standards. In short, users will input their desired specifications, while the model will output the expected circuit parameters. However, in the preliminary stage, an ANN model has been developed to predict the device sizes of a fully differential telescopic cascode two-stage voltage amplifier (TSVA) as a proof-of-concept to validate the feasibility of the proposed system. Finally, the trained model has achieved a test loss of 0.2322 and a validation loss of 0.2418 in only 60.78 seconds. This project will benefit analogue circuit designers in the sense that it saves time in predicting circuit parameters so that they can focus on other aspects of the design.

ABSTRAK

Kajian ini dilaksanakan untuk memperkenalkan sesuatu alat cipta reka elektronik (EDA) untuk mengoptimasikan reka bentuk litar analog. Hal ini akan dicapai melalui rangka “Artificial Neural Network” yang akan meramal nilai-nilai transistor terdapat dalam litar tersebut. Selanjutnya, ANN akan dilatih dengan data hasil daripada OCEAN dalam Cadence. Sebenarnya, dua model ANN akan dihasilkan, iaitu: “performance model” dan “optimization algorithm”. Fokus projek ini melibatkan *op-amp* berbilang peringkat, termasuk *op-amp* tiga peringkat iaitu Reversed Nested Miller Compensation (RNMC), supaya kekal relevan dengan piawaian industri terkini. Secara ringkas, pengguna akan memasukkan spesifikasi yang diinginkan, manakala model ini akan mengeluarkan parameter litar yang dijangkakan. Walau bagaimanapun, pada peringkat awal, satu model ANN telah dibangunkan untuk meramal saiz peranti bagi “fully differential telescopic cascaded two-stage voltage amplifier” (TSVA) sebagai bukti konsep bagi mengesahkan kebolehlaksanaan sistem yang dicadangkan. Akhir sekali, model terlatih tersebut telah mencapai “*test loss*” sebanyak 0.2322 dan “*validation loss*” sebanyak 0.2418 dalam 60.28 saat Sahaja. Projek ini akan memanfaatkan perekabentuk litar analog dengan menjimat masa agar mereka dapat fokus pada aspek penting lain.

TABLE OF CONTENTS

	TITLE	PAGE
	DECLARATION	iii
	ACKNOWLEDGEMENT	v
	ABSTRACT	vi
	ABSTRAK	vii
	TABLE OF CONTENTS	viii
	LIST OF TABLES	x
	LIST OF FIGURES	xi
	LIST OF ABBREVIATIONS	xiii
	LIST OF APPENDICES	xv
CHAPTER 1	INTRODUCTION	1
1.1	Problem Background	1
1.2	Problem Statement	2
1.3	Research Objectives	2
1.4	Research Scope	2
1.5	Paper Structure	3
CHAPTER 2	LITERATURE REVIEW	4
2.1	Theoretical Review	4
2.2	Review on Machine Learning	4
2.3	Supervised Machine Learning	5
2.4	Unsupervised Machine Learning	6
2.5	Artificial Neural Network (ANN)	7
2.6	Two Stage Voltage Amplifiers (TSVA)	11
2.7	Compensation Techniques	13
2.8	Reversed Nested Miller Compensation (RNMC)	14
2.9	Gate transconductance efficiency (g_m / I_d) based Methodology	16

2.10	Automated Design of Analogue Circuits using Machine Learning Techniques	18
2.11	Optimizing Analogue Circuit Design Through a Machine Learning Assisted Evolutionary Algorithm (EA)	19
2.12	Analogue Integrated Circuit Topology Synthesis with Deep Reinforcement Learning	20
2.13	MACRO: Multi-agent Reinforcement Learning-based Cross-layer Optimization of Operational Amplifier	21
2.14	Tabular Comparison of Relevant References	21
CHAPTER 3	RESEARCH METHODOLOGY	25
3.1	Proposed Method	25
3.2	Tools and Platforms	28
3.3	Project Management	28
CHAPTER 4	RESULTS AND DISCUSSION	30
4.1	Proof of Concept	30
4.2	Results and Discussion	36
4.3	Chapter Summary	37
CHAPTER 5	CONCLUSION AND RECOMMENDATIONS	39
5.1	Conclusions	39
5.2	Contributions to Knowledge	39
5.3	Future Works	40
	REFERENCES	41

LIST OF TABLES

TABLE NO.	TITLE	PAGE
Table 2.1:	Comparison of different machine learning tasks	6
Table 2.2:	Common activation functions in ANN	8
Table 2.3:	Common optimizer functions	10
Table 2.4:	Comparison of Miller compensation and RAFFC	16
Table 2.5:	Comparison between methods in [3]	18
Table 2.6:	Comparison between past methodologies	22
Table 3.1:	Cost of project	29
Table 4.1:	Table showing inputs and outputs of ANN model	36

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
Figure 2.1:	Context of different artificial intelligence concepts	4
Figure 2.2:	Comparison between classification and regression	5
Figure 2.3:	Figure of neuron in an ANN	7
Figure 2.4:	Architecture of ANN	9
Figure 2.5:	Block diagram of two stage amplifier	11
Figure 2.6:	Schematic of telescopic cascode differential amplifier	11
Figure 2.7:	Common source amplifier	12
Figure 2.8:	Connection of compensated TSVA	13
Figure 2.9:	Bode plot for gain and phase	14
Figure 2.10:	Block diagram of basic RNMC	14
Figure 2.11:	RAFFC configuration in RNMC op-amps	15
Figure 2.12:	g_m / I_d vs Normalized current, I_N for $L = 60\text{nm}$ (dotted) and $6\mu\text{m}$ (solid) at $V_{DS} = 0.2, 0.5$ and 1.0 V	17
Figure 2.13:	Graph of cost against number of runs for EA with DNN (blue) and EA without DNN (orange) as extracted from [6]	19
Figure 2.14:	DRL operation as extracted from [23]	20
Figure 3.1:	Overview of proposed system	26
Figure 3.2:	Flowchart of training an ANN model	27
Figure 3.3:	Structure of an ANN	27
Figure 3.4:	Gantt chart showing project schedule	28
Figure 3.5:	Gantt chart for upcoming schedule	29
Figure 4.1:	Schematics of two-stage voltage amplifier circuit	30
Figure 4.2:	Netlist of TSVA generated by Cadence	31
Figure 4.3:	OCEAN script used in generating data	32
Figure 4.4:	Dataset generated by OCEAN scripting	33
Figure 4.5:	Code snippet for ANN model	35

Figure 4.6: Test loss of preliminary ANN model	36
Figure 4.7: Training loss of preliminary ANN model	36
Figure 4.8: Terminal output showing elapsed time for ANN training	37

LIST OF ABBREVIATIONS

ADAM	-	Adaptive Moment Estimation
ANN	-	Artificial Neural Network
CMFB	-	Common-Mode Feedback
CMOS	-	Complementary Metal-Oxide-Semiconductor
DAC	-	Digital-to-Analogue Converter
DIBL	-	Drain-Induced Barrier Lowering
EA	-	Evolutionary Algorithm
EDA	-	Electronic Design Automation
FOC	-	Free of Charge
FoM	-	Figure-of-Merit
GA	-	Genetic Algorithm
GBW	-	Gain-Bandwidth Product
GNN	-	Graph Neural Network
IP	-	Intellectual Property
LHP	-	Left-Half Pole
MARL	-	Multi-agent Reinforcement Learning
ML	-	Machine Learning
MLP	-	Multilayer Perceptron
MSE	-	Mean Squared Error
Op-amp	-	Operational Amplifier
OTA	-	Operational Transconductance Amplifier
PE	-	Processing Element
PSO	-	Particle Swarm Optimisation
PT	-	Parameter Tuning
RAFFC	-	Reversed Active Feedback Frequency Compensation
ReLU	-	Rectified Linear Unit
RHP	-	Right-Half Pole
RL	-	Reinforcement Learning
RMSprop	-	Root Mean Square Propagation
RNMC	-	Reversed Nested Miller Compensation

SGD	-	Stochastic Gradient Descent
SR	-	Slew Rate
TD	-	Topology Design
TSVA	-	Two Stage Voltage Amplifier
VAE	-	Variational Autoencoder

LIST OF APPENDICES

APPENDIX	TITLE	PAGE
No table of figures entries found.		

CHAPTER 1

INTRODUCTION

1.1 Problem Background

Currently, digital circuits are greatly emphasized for their importance in the field of electronics [1]. Hence, there exists a lot of EDA tools for digital circuit design. However, the same cannot be said for analogue circuit designs [2]. The challenge mainly arises from the non-linear characteristics of the circuit parameters, making them hard to predict. More often than not, optimizing one parameter may result in the degradation of another [3]. Thus, multiple iterations are required to optimize the circuits to achieve desired specifications, wasting precious time in simulation and validation processes [4-6]. Trade-offs must be made between the performance metrics to achieve the best performance [7]. As such, a quicker way to predict transistors' sizing must be proposed to speed up the time-to-market of analogue circuits [8]. Although researchers have made efforts to automate different stages in the design flow [1, 9] this project focuses on expediting the circuit sizing process.

Simultaneously, with rapid advancements in the field of electronics, supply voltages for circuits continue to scale down, making conventional cascode topologies unsuitable anymore [10]. This is because cascoding transistors reduce voltage swing which leads to lower gain. So, high gain and high bandwidths are instead achieved by cascading multiple stages of Operational Transconductance Amplifiers (OTA). Among the most used topologies include the three-stage Reversed Nested Miller Compensation (RNMC) design, which will be the focus of this project. The RNMC is a nested Miller Compensation technique that exploits the Miller effect to split slow frequency poles to achieve desired phase margin and transient response [11-14]. However, to validate the proposed framework, a preliminary study will be done on a two-stage voltage amplifier (TSVA) using Cadence and OCEAN scripting.

1.2 Problem Statement

Despite advancements in EDA tools, it is clear that designing analogue circuits remains a manual process heavily relying on human validation and experience [15]. This is especially the case for multi-stage analogue circuits, such as the TSVA and the RNMC configuration op-amps. Complex trade-off between gain, bandwidth and power make the process of optimizing a circuit too time-consuming, which stems from the circuit parameters' high degree of non-linearity [16]. This creates a critical need for an automated, machine-learning-driven approach to predict optimal device sizing without excessive simulation iterations.

1.3 Research Objectives

The ultimate aim of this project is to create an EDA tool for optimizing analogue circuits designs. The objectives in this project are:

- (a) To design an ANN machine learning (ML) model for predicting transistors' sizing in an analogue circuit.
- (b) To reduce time consumption in analogue circuit design.
- (c) To benchmark and assess the performance of the ANN model prediction.

1.4 Research Scope

The scope of this project focuses on the three-stage RNMC operational amplifiers (op-amps), mainly due to their high complexity and their ability to drive capacitive loads. Successfully optimizing the RNMC topology serves as a solid benchmark to validate the framework's scalability for other complex analogue circuits. Hence, an ANN model will be utilised as the core inference engine, which will be developed with Python and its relevant libraries, including but not limited to PyTorch and TensorFlow. Moreover, BSIM4 (Level 54) transistor model will be applied for all devices to accurately simulate submicron processes.

1.5 Paper Structure

The paper is organized as follows. The literature review is shown in Chapter 2 which explores multiple concepts regarding the use of machine learning on optimizing circuit design. Proposed methodology is shown in Chapter 3 which illustrates the proposed framework in detail. Chapter 4 shows some of the preliminary data produced and analysis on the results obtained. A brief conclusion and future improvements are collected in Chapter 5.

CHAPTER 2

LITERATURE REVIEW

2.1 Theoretical Review

A literature review is done to explore relevant concepts and methods that are used for predicting analogue circuit sizing. Each concept will be elaborated in detail.

2.2 Review on Machine Learning

As illustrated in Figure 2.1, machine learning is a subset of artificial intelligence where computer systems learn from data, recognize patterns and make decisions without being explicitly programmed to do so [17]. Arthur Samuel described it as “the ability of computers to learn without programming new skills directly”.

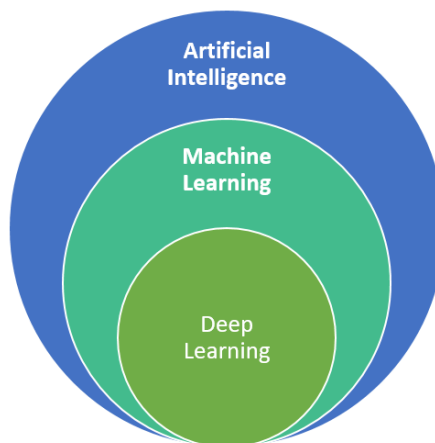


Figure 2.1: Context of different artificial intelligence concepts

Machine learning algorithms can be said to be made up of mathematical equations, which are capable of generating predictions/decisions based on available data.

Among the main types of machine learning include: supervised and unsupervised learning.

2.3 Supervised Machine Learning

Supervised machine learning is where the model is trained on dataset that relates to the problem at hand, i.e., consisting of both input and output data. The model is fully aware of the golden standard – “ground truth” of the provided data. Once the model learns the relationship between the input and output data, it is able to classify new unknown datasets and make corresponding predictions. This type of supervised learning consists of: **classification and regression**. In short, classification is a binary decision; yes or no while regression predicts continuous values using various input data. For example, classification is for classifying between cats and dogs while regression is for predicting house prices based on size and location. Figure 2.2 below shows the comparison between classification and regression.

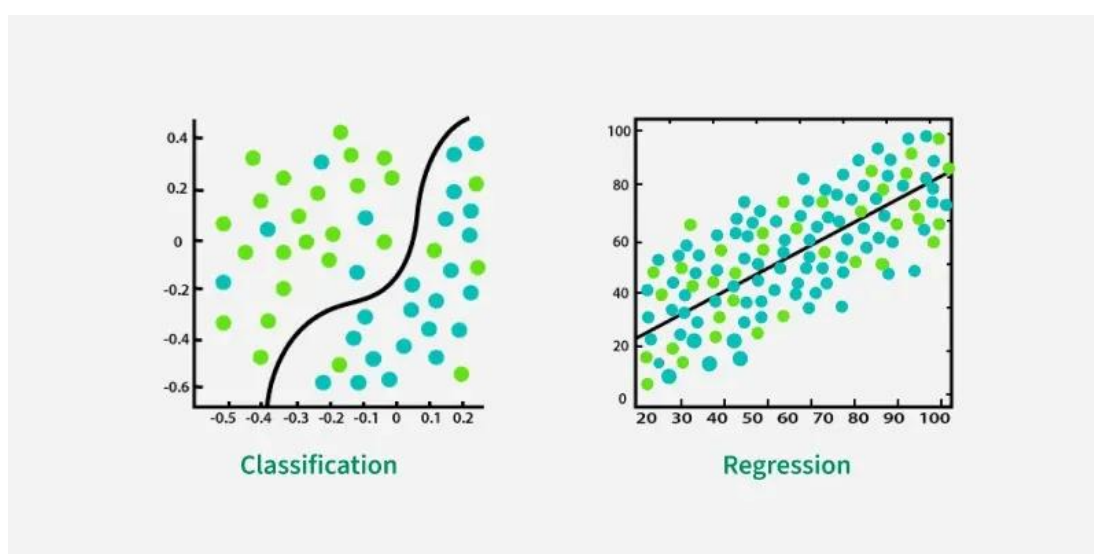


Figure 2.2: Comparison between classification and regression

2.4 Unsupervised Machine Learning

Conversely, unsupervised machine learning differs from supervised learning in the sense that the training data is unannotated (not labelled by humans or algorithms) [17]. The dataset do not provide answers to the given tasks, and it is up to the machine learning model to distinguish data based on common characteristics. In unsupervised learning, the ultimate goal is to enable the model to detect patterns and group data without a provided answer. There are also two types of unsupervised learning: **clustering and association**. In short, clustering groups data based on similarities and differences while association analyses the relationships between data in a dataset. For example, clustering is able to cluster animals with similar features while association maps how likely it is for someone to purchase bread to also purchase butter.

A comparison of different machine learning tasks is shown in Table 2.1 below. Hence, it becomes clear that regression is the most suitable candidate to predict non-linear circuit parameters in op-amps. Also, supervised learning is the most appropriate method as valid labelled datasets will be generated through scripting in Cadence.

Table 2.1: Comparison of different machine learning tasks

Classification	Regression	Aspects	Clustering	Association
Supervised	Supervised	Machine Learning Type	Unsupervised	Unsupervised
Yes	Yes	Labels required	No	No
Discrete class labels	Continuous numerical values	Output	Clusters of similar items	Rules between items

2.5 Artificial Neural Network (ANN)

Artificial Neural Networks (ANNs) are a fundamental type of machine learning which are artificial adaptive systems inspired by the functioning process of the human brain [18]. ANNs are able to modify their internal structure in relation to a function objective, making them suitable for solving non-linear problems with the ability to reconstruct fuzzy rules that govern the optimal solution to problems. Basic element of an ANN include the nodes/processing elements (PE). Each node has its own input, which is connected to other nodes or the environment. Then, each node also has a function, f that transforms input to output. Briefly, this relation is depicted in Figure 2.3 below.

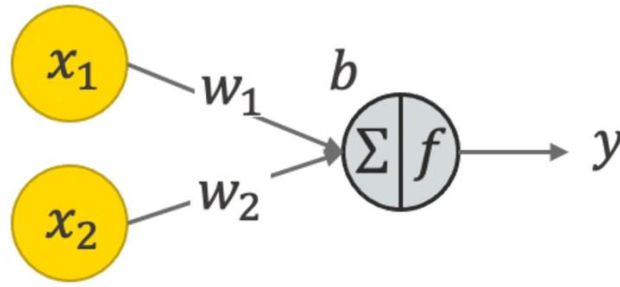


Figure 2.3: Figure of neuron in an ANN

The relationship of the nodes can be expressed mathematically as:

$$y_i = \sum_{i=0}^n w_i x_i + b_i \quad (2.1)$$

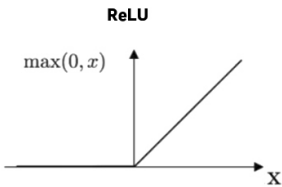
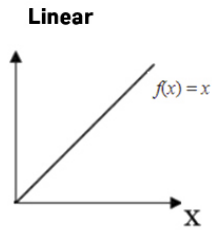
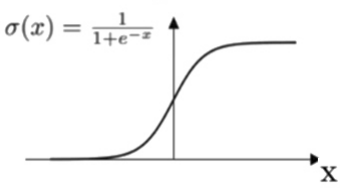
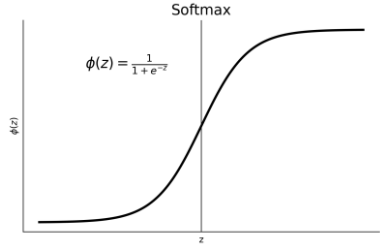
Where y is the output value, w is the weight of the connections from previous nodes or from the environment, x is the output of previous nodes, and b is the bias values applied to each layer. For weight, w , positive values indicate excitatory connections while negative values indicate inhibitory connections. By modifying these weight values, ANNs are able to update themselves through the ‘Law of Learning’ [18]. However, equation (2.1) is only suitable for linear applications, which is not often the case for

complex problems. Hence, activation functions are introduced to allow ANNs to model non-linear problems. The complete operation in a PE of ANN is expressed in Equation 2.2 below:

$$y_i = f(\sum_{i=0}^n w_i x_i + b_i) \quad (2.2)$$

Where the function, f is known as the activation function to model complex non-linear characteristic in the given problem. Some of the common activation functions are compiled in Table 2.2 below.

Table 2.2: Common activation functions in ANN

Activation function	Range	Application	Graph
Rectified Linear Unit (ReLU)	$[0, \infty)$	Hidden layers	
Linear	$(-\infty, \infty)$	Output layer	
Sigmoid	$(0,1)$	Binary classification	
Softmax	$(0,1)$	Multi-class classification	

With that, the general architecture of a feed-forward ANN consists of an input layer, one/multiple hidden layer(s), and a final output layer. Each layer contains interconnected nodes. The illustration is shown in Figure 2.5 below.

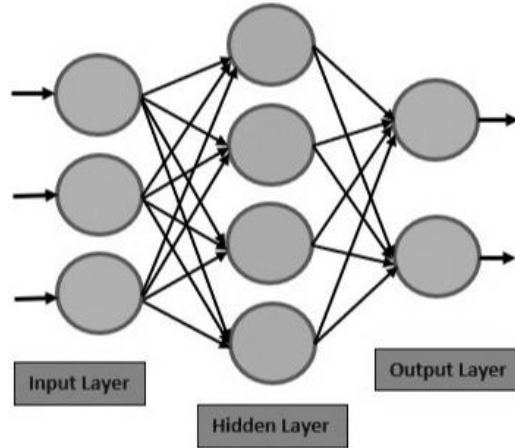


Figure 2.4: Architecture of ANN

In feed-forward neural networks, data flows in one direction from input to output. The input layer receives raw data from provided datasets, i.e. the design parameters (width and length of transistors). The hidden layers are the intermediary layers that consist of activation functions to extract features or model relationship between data. Finally, the output layer produces the final prediction. This forward pass is also known as a performance model that maps input data to output specifications [4].

But how does the model actually “learn”? In contrast to feed-forward, a backward propagation is implemented to update the individual weights, w for each connections. A loss function compares the model’s prediction and the ground truth where the most common metric is the Mean Squared Error (MSE). Then, the gradient is calculated by taking the derivative of the loss with respect to the weights to determine the direction in which the weights must be adjusted to minimise error. Finally, an optimizer will subtract a fraction of the gradient from the original weights to update the respective nodes. Mathematically, the backwards propagation method can be expressed as:

$$W_{new} = W_{old} - (learning\ rate \times \frac{\partial Loss}{\partial w}) \quad (2.3)$$

Where W_{new} is the new weight, $\frac{\partial \text{Loss}}{\partial W}$ is the gradient and W_{old} is the old weight. The learning rate can be thought of as “step size” in updating the hyperparameters. If the learning rate (step size) is too small, the model will require a long time to reach the solution (converge). Whereas large learning rate may cause the model to miss the solution (diverge). Hence, the learning rate is critical to ensure best model performance in ANNs which is determined by the optimizer algorithm. Some of the commonly used optimizers are shown in Table 2.3 below.

Table 2.3: Common optimizer functions

Optimizer	Operation
Adaptive Moment Estimation (Adam)	Computes individual adaptive learning rates for different parameters based on mean and variance of gradients.
Stochastic Gradient Descent (SGD)	Updates weights using gradient of loss function of a subset of data rather than entire dataset.
Root Mean Square Propagation (RMSprop)	Keeps a moving average of squared gradients to adjust learning rates.

2.6 Two Stage Voltage Amplifiers (TSVA)

After covering the basics of machine learning, it is time to cover the fundamentals of analogue circuits. As its name suggest, TSVA comprises of two cascaded op-amps, where its block diagram is shown below in Figure 2.5:

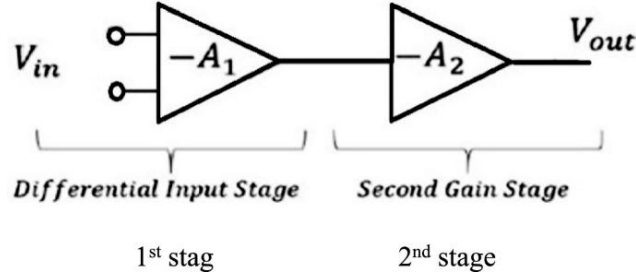


Figure 2.5: Block diagram of two stage amplifier

The main advantage of cascading multiple stages of amplifier is to overcome the limitations of single-stage or cascode topologies in achieving high gain and high swing [19]. So, the requirements are separated and tackled in each of the multi-stage op-amps. Specifically in TSVA, the first stage is normally a differential input amplifier that is designed to provide high DC gain while providing good noise immunity [20]. The schematic of a telescopic cascode differential amplifier is shown in Figure 2.6.

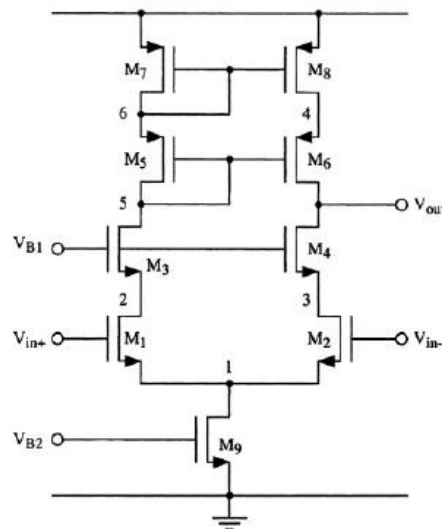


Figure 2.6: Schematic of telescopic cascode differential amplifier

In the first stage, the high gain is achieved via the cascode configuration of the PMOS load, which significantly increases the effective output resistance of the differential amplifier. From [20], the DC gain expression is given as:

$$A_{v1} = (g_{mi} \times R_{outi}) \quad (2.4)$$

Where A_{v1} is the DC gain, g_{mi} is the transconductance of the input transistor and R_{outi} is the effective output resistance of the output network. Hence, the increased output resistance directly leads to an increase in DC gain.

For the second stage, it is usually a common source amplifier with active load that is designed to provide large output swing [19]. This allows the output voltage to swing closer to the supply rails, i.e. V_{DD} and V_{SS} . This is because the simple transistors do not require large voltage headroom to operate in the saturated region. The common source amplifier is shown in Figure 2.7 below.

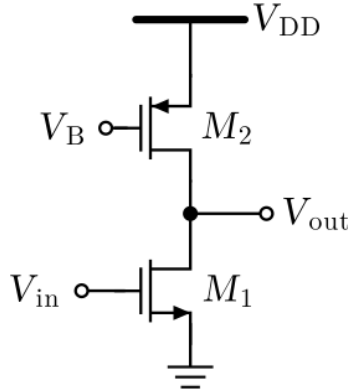


Figure 2.7: Common source amplifier

The input transistor can be the NMOS or PMOS depending on the application. Usually when used as the second stage amplifier, the PMOS is the input while the NMOS is the load. Thus, it is clear that by cascading both configurations of amplifiers, high gain and high swing are achieved. However, the addition of amplification stage introduces an additional pole in a closed loop system that causes the phase of the system to drop drastically [20]. As a result, oscillation and ringing effects degrade the performance of the circuit. This is solved by compensation techniques that aim to improve the phase margin of the system.

2.7 Compensation Techniques

Compensation of multi-stage op-amps functions to improve stability by preventing oscillation and improve phase margin of the system. A phase margin of 60° is generally considered as optimal [20]. Compensation is achieved by connecting a capacitor from the input of 2nd stage to the output of 2nd stage. The connection is illustrated in Figure 2.8 for a better visual representation.

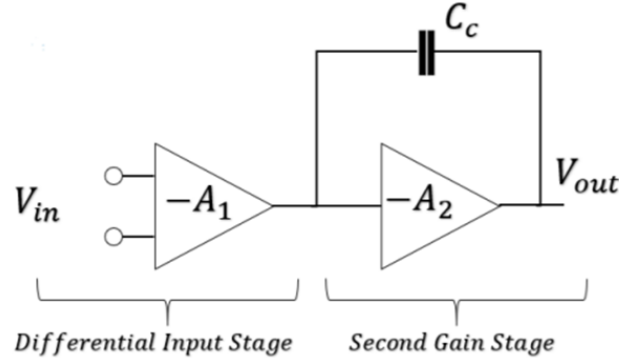


Figure 2.8: Connection of compensated TSVA

Adding the compensation capacitor, C_c enables pole splitting that leverages the Miller effect to modify the frequency response by separating the low frequency poles. The large Miller capacitance at the first stage's output creates a new dominant pole at a low frequency, causing gain to roll off earlier, ensuring that the gain crossover occurs earlier than phase crossover [20]. This condition ensures the stability of the system. Generally, the poles and zeros of a system is represented as:

$$G(s) = \frac{(1 + \frac{s}{z})}{(1 + \frac{s}{p_1})(1 + \frac{s}{p_2})} \quad (2.5)$$

Where z is the system zero and p_1 and p_2 are the system poles. The relationship between the gain and phase response is clearly seen in the bode plots as shown in Figure 2.9:

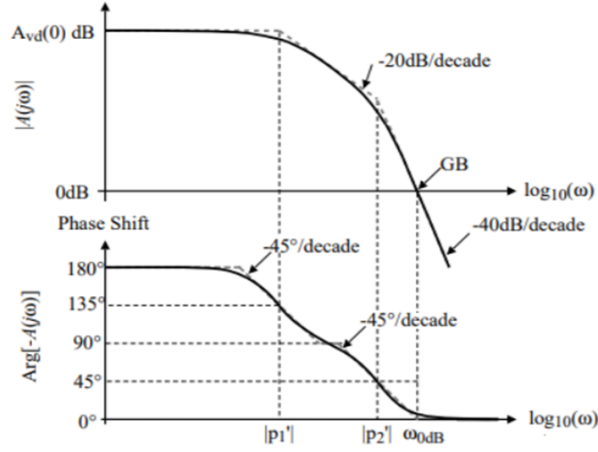


Figure 2.9: Bode plot for gain and phase

From Figure 2.9, it becomes clear that the phase margin must be greater than -180° when the gain crosses the 0dB line to ensure system stability, which is achieved with the compensation capacitor, C_C .

2.8 Reversed Nested Miller Compensation (RNMC)

As supply voltage continues to scale down, traditional cascode topologies are no longer suitable for achieving high gain and swing [10]. The same goes for the TSVA as presented in Section 2.6. As such, three stage amplifiers have been introduced to tackle the aforementioned issue of low voltage supply. The common compensated three stage amplifier is the Reversed Nested Miller Compensation (RNMC) topology, which is shown in Figure 2.10 below.

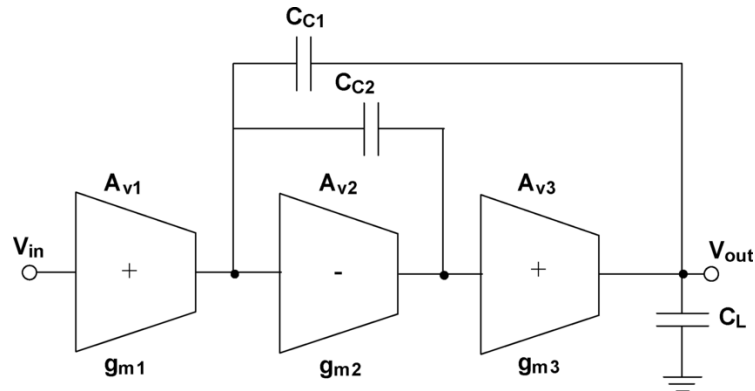


Figure 2.10: Block diagram of basic RNMC

From [10], the RNMC topology is capable of driving heavy capacitive loads such as high-accuracy modulators, flash and pipeline analogue-to-digital converters (DAC), linear regulators and active matrix display drivers. As seen in Figure 2.10, the second stage is the only inverting stage, hence the term reversed NMC. It exploits the same Miller effect as the compensation capacitor, C_C in Section 2.7, but shows inherent bandwidth improvements [10] as the inner capacitor does not load the output node [12].

Among RNMC, the Reversed Active Feedback Frequency Compensation (RAFFC) is considered superior to other compensation strategies as proven in [13]. Its major advantage is that it can be implemented without entailing extra transistors, thus keeping the circuit complexity low whilst maintaining high performance. The RAFFC consists of a current buffer, g_{mb} in the outer loop as shown in Figure 2.11 below.

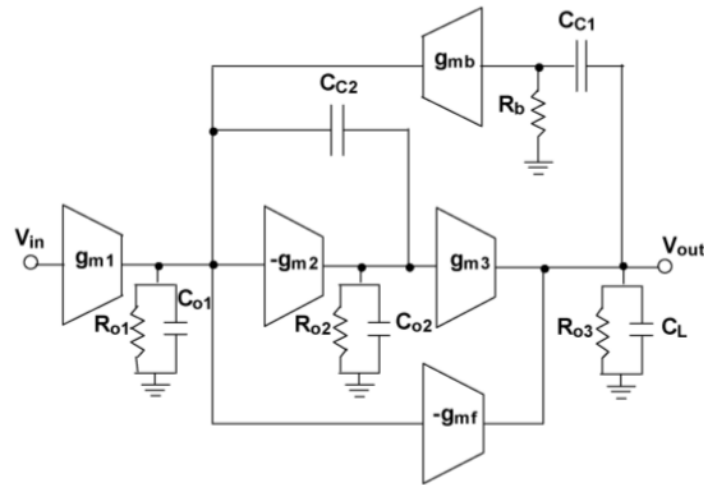


Figure 2.11: RAFFC configuration in RNMC op-amps

Briefly, RAFFC provides higher gain-bandwidth product (GBW) and better phase margin compared to traditional compensation techniques, i.e. Miller compensation. RAFFC introduces a Left-Half-Pole (LHP) zero that cancels out non-dominant poles, therefore allowing extended bandwidth without affecting stability. Furthermore, RAFFC also enhances slew rate, enabling the amplifier to drive highly capacitive loads

efficiently. A simple comparison of RAFFC and Miller compensation is made in Table 2.4.

Table 2.4: Comparison of Miller compensation and RAFFC

Compensation	Miller	RAFFC
Zero type	RHP – unstable	LHP – stable
Phase margin	Medium	High
Required capacitance	High	Small
Gain-bandwidth product (GBW)	Small	High
Output swing	Reduced	Preserved
Suitability for 3-stage	Poor	Excellent

Hence, it becomes clear that RAFFC is the better candidate for three-stage op-amps and this specific topology will be the end goal of this project.

2.9 Gate transconductance efficiency (g_m / I_d) based Methodology

Per the prediction of Moore’s Law, the number of transistors on chips have increased exponentially and correspondingly, transistor sizes have also decreased significantly. In fact, transistors have already been reduced to the nanometer range, where short channel effects become dominant. Hence, the square law for predicting transistor sizing is no longer valid as they are only restricted to long channel devices [21]. As such, the g_m / I_d method is used to size transistors instead. Transconductance efficiency, g_m / I_d shows the gain capability per unit of bias current (power consumption), and for a single dominant pole and capacitive load, C_L , it can be expressed as:

$$\frac{g_m}{I_d} = \frac{GBW}{SR} = \frac{\frac{g_m}{C_L}}{\frac{I_d}{C_L}} \quad (2.6)$$

Which is a ratio of gain-bandwidth product (GBW) and the slew rate (SR).

The g_m / I_d method provides a simple way for circuit designers to calculate the required aspect ratio (W/L) of transistors to meet specific performance goals. This is because g_m / I_d provides a unique one-to-one mapping between the transistors' operating inversion region to the dimensions, allowing designers to make optimum trade-offs between many figure-of-merits (power, speed, gain and noise). It replaces the conventional hand calculations by providing a graphical, lookup-table based design methodology. An example of g_m / I_d graph is shown in Figure 2.12 below:

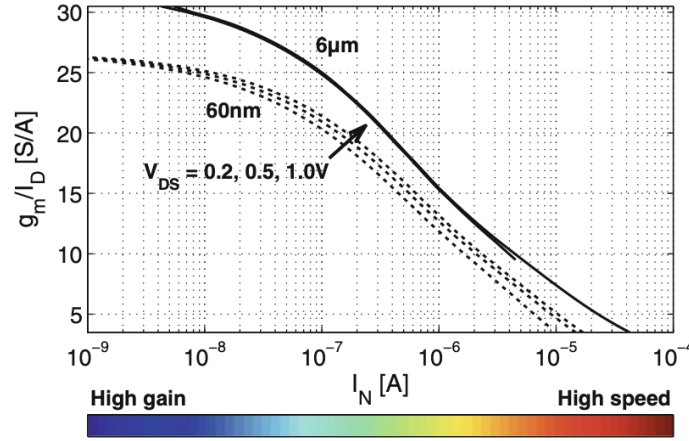


Figure 2.12: g_m / I_d vs Normalized current, I_N for $L = 60nm$ (dotted) and $6\mu m$ (solid) at $V_{DS} = 0.2, 0.5$ and 1.0 V

Briefly, designers will decide whether to target high speed or power and determine the corresponding g_m / I_d . Then, based on g_m / I_d values from the graph, transistors' width are calculated based on the predetermined power consumption, I_d .

Furthermore, authors from [22] have proven that transistor characteristics are also dependent on source-drain voltage, V_{DS} and length, L of the device. Hence, characterizing the transistor becomes a function of g_m / I_d , V_{DS} and L , to account for Drain-Induced Barrier Lowering (DIBL) effects in deeply scaled CMOS technologies. The g_m / I_d method will be used in the proposed methodology as a preprocessing step of dataset to ensure valid data.

2.10 Automated Design of Analogue Circuits using Machine Learning Techniques

From paper [3], a multilayer perceptron (MLP) ANN machine learning model has been deployed to automate the design of analog circuits. The number of input neurons corresponds to the number of input parameters, for example, gain, current and gain-bandwidth product (GBW). Similarly, the number of output neurons depend on the design parameters to be estimated, i.e., transistor widths, lengths and resistances. As there are multiple input and output parameters, the use of MLP is justified.

Then, backward propagation was completed to predict circuit parameters based on the specifications. Conversely, forward propagation was done to validate that ANNs are capable of predicting specifications from input parameters, functioning just like a SPICE simulator. Instead of randomly sweeping the transistor width (W) and length (L), the innovative methodology lies in utilizing the g_m / I_d technique to reduce the size of dataset. Despite the smaller dataset size, the produced ANN model has shown a score of 93% accuracy. Table 2.4 below shows the comparison between methods extracted from [3] performed on a two-stage single-ended op-amp.

Table 2.5: Comparison between methods in [3]

Method	Dataset	Score (%)
Global ANN	45583	30.2
Local ANN with GA	632	75.8
Sweeping W & L	65534	90
g_m / I_d	7361	93

Hence, it is evident that utilising the g_m / I_d method yields a smaller dataset that produces a highly accurate model. As such, this method will be adopted in this project as well.

2.11 Optimizing Analogue Circuit Design Through a Machine Learning Assisted Evolutionary Algorithm (EA)

Reference [6] core innovation lies in the integration of Deep Neural Networks (DNN) in the evolutionary algorithm (EA) to predict whether a produced offspring is high-performing. While EAs are effective at finding optimal designs, they require thousands of SPICE simulation to determine their fitness, making them computationally expensive. Hence, a DNN has been integrated to predict whether a crossover is “fit” enough to be kept, thereby effectively filtering out bad designs and accelerating the convergence process. The DNN performs binary classification with the sigmoid activation function which eliminates the need of expensive SPICE simulation during each iteration update of circuit parameters. This methodology was tested on a two-stage op-amp circuit using 40nm CMOS technology, which converged in only 260 runs as compared to normal EA without DNN integration. The graph of cost against number of runs is presented in Figure 2.13 below.

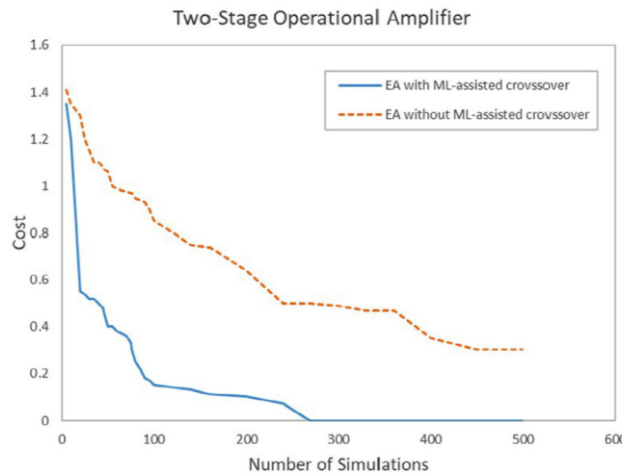


Figure 2.13: Graph of cost against number of runs for EA with DNN (blue) and EA without DNN (orange) as extracted from [6]

It successfully showed that the integration of ML in EA greatly reduces the computation time and power needed to predict circuit parameters.

2.12 Analogue Integrated Circuit Topology Synthesis with Deep Reinforcement Learning

Unlike Sections 2.10 – 2.11, paper [23] focuses on the domain of topology synthesis, by creating new schematic structure with op-amps. However, this is much more difficult when compared to the sizing problem, as the design space is too large, with endless possibilities. Nonetheless, the authors have provided common circuits as building blocks for the Deep Reinforcement Learning (DRL) model to synthesize new topologies. With that, strict topological constraints were also defined to prevent the DRL from creating illogical circuits such as shorting V_{DD} to Gnd. The DRL learns from reward functions that are determined by the targeted performance. As a result, the DRL has successfully rediscovered the two-stage miller op-amp circuit without being explicitly taught in only hours, which is comparable to state-of-the-art evolutionary methods. Furthermore, hash tables were also employed to speed up evaluations on generated circuits to avoid duplicates. In short, the DRL is a ML model fully capable of creating valid innovative circuits that achieve targeted specifications. Figure 2.14 below shows the operation of the proposed framework and the interaction between the DRL and the usage of hash tables for circuit evaluation.

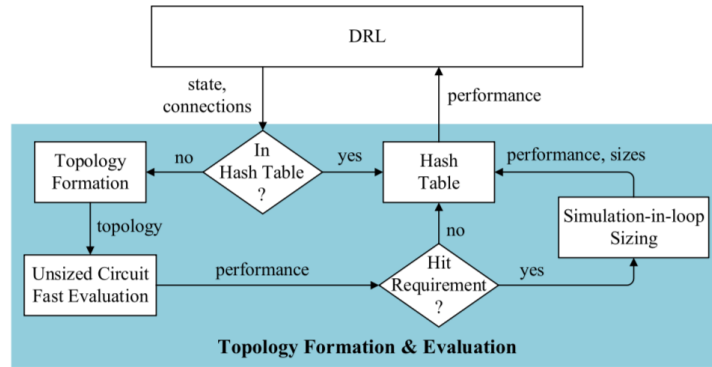


Figure 2.14: DRL operation as extracted from [23]

2.13 MACRO: Multi-agent Reinforcement Learning-based Cross-layer Optimization of Operational Amplifier

Paper [8] introduces a new concept known as cross-layer optimization that involve both agents: Agent T and Agent P to tackle topology design (TD) and parameter tuning (PT) problems. Conventionally, these two domains were isolated from each other as TD is discrete and qualitative while PT is continuous and quantitative. As such, conventional approach was slow to converge as each process was a ‘black box’. In contrast, with MACRO, both TD and PT are automated simultaneously using Multi-agent Reinforcement Learning (MARL), allowing both agents to co-evolve and learn how changes in topology affect sizing and vice versa. As a result, MACRO generated three-stage op-amps with figure-of-merit (FoM) that was 2.22 times higher than Bayesian baseline and 1.27 times higher than the standard reinforcement learning (RL) baseline. It effectively proves that MACRO is practical in real use cases and are capable of generating higher quality circuits with the sharing of information across different ML models.

2.14 Tabular Comparison of Relevant References

After covering all the relevant concepts from Section 2.1 – 2.14, important methodologies are compared in Table 2.6 below.

Table 2.6: Comparison between past methodologies

Paper	Summary	Advantages	Disadvantages
Automated Design of Analog Circuits using Machine Learning Techniques [3]	Inverse Design (Supervised Learning): Uses a Global ANN to directly map performance specifications (Gain, Power) to circuit parameters (W, L). The model acts as the "Designer."	Instant Results: Design generation takes milliseconds once trained. Spice-Free: No simulations needed during the design phase. Good accuracy (93.44%).	Data Hungry: Requires a massive pre-existing dataset (7,361 samples) to train effectively. Only solves sizing, not topology.
Optimizing Analog Circuit Design Through a Machine Learning-Assisted Evolutionary Algorithm [6]	Evolutionary Algorithm (EA) with ML: Uses a Deep Neural Network (DNN) as a "Guide" to predict if a genetic crossover will result in a better design	High accuracy (>92% classification) Generates data dynamically, not requiring a static dataset.	Slow runtime, requiring 8 hours per run. Computationally expensive, as it requires in-loop SPICE simulations
Analog Integrated Circuit Topology Synthesis With Deep	Deep Reinforcement Learning (DRL): Uses a Policy Gradient Neural Network to act as an	Creative: Can synthesize new circuit structures (topologies) from scratch.	Complex Setup: Training takes 2–9 hours. Harder to implement than

Reinforcement Learning [23]	"Architect," building the circuit topology step-by-step and learning from rewards.	Very high success rate (99%). Uses very few actual SPICE simulations (20-80).	standard regression. Requires complex reward function definition.
MACRO: Multi-agent Reinforcement Learning-based Cross-layer Optimization of Operational Amplifier [8]	Multi-Agent RL (Cross-Layer): Uses two interacting agents (one for Topology, one for Sizing) that co-evolve to optimize the entire design flow simultaneously	Holistic: Prevents choosing a topology that is impossible to Flexible: Handles discrete (topology) and continuous (size) spaces well.	Very Complex: Requires training 2 agents + VAE + GNNs. Computationally Expensive: Needs GPU and extensive training loops. Black Box: Hard to interpret why agents chose a specific path
Design Optimization of Analog Integrated Circuits by Using Artificial Neural Networks [16]	Neural Network Optimization: An early Inverse Design approach using a simple Multilayer Perceptron (MLP) to approximate circuit equations and predict sizes from specs.	Fast Training: Converges in just 134 runs. Simple: Easy to implement standard MLP architecture. Eliminates the need for manual hand calculations.	Small Data: Uses a tiny dataset (200 samples) which risks overfitting. Outdated: Lower precision standards (Error < 10^{-2}) compared to modern Deep Learning methods.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 Proposed Method

Usually, a **feed-forward** ANN will be used as the **performance model** to map design parameters to circuit performance[24-27]. Meanwhile, an **optimization algorithm** will implement a **backward propagation** technique to find design parameters based on input design goal [25, 28-33].

This project will take a similar approach by implementing a multilayer perceptron (MLP) ANN to tackle multiple input and output simultaneously. However, inverse design will be implemented to solve sizing based on targeted specifications. The number of nodes in hidden layers will be optimised to ensure high performance without overfitting the model. Any overfitting will be addressed using L2 weight regularisation [2]. Also, all layers are fully connected to allow the modelling of complex non-linear relationships between the parameters. An overview of the proposed system is shown in Figure 3.1.

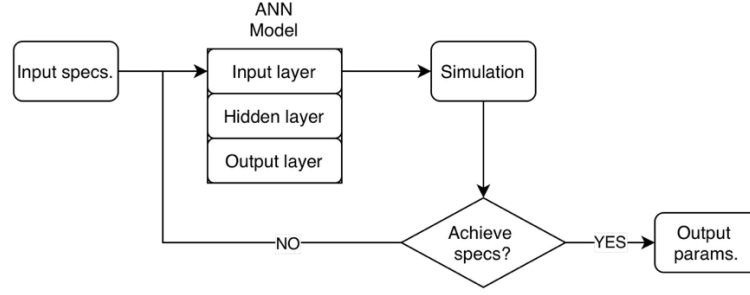


Figure 3.1: Overview of proposed system

As seen from Figure 3.1, the desired specifications of the circuit, i.e. gain, bandwidth, power, etc are input to the MLP ANN model. Like any general ANN, the MLP consists of input layers, hidden layers and output layers. The number of input nodes corresponds to the number of input specification, while the number of output nodes reflects that of the output circuit parameters. Then, the produced results will be benchmarked against off-the-shelf simulators to validate their accuracy. If performance falls short, the correct values will be added to the dataset to update the ANN model for better accuracy.

The proposed method has an advantage compared to conventional approach utilizing **geometric programming framework** (posynomial and monomial functions) and **Particle Swarm Optimisation** (PSO) which are prone to deviate from real circuit performance[30]. Its advantage lies in the fact that ANNs are able to model continuous non-linear functions of multiple inputs simultaneously [18]. However, ANNs are only able to predict parameters for a fixed topology unlike genetic algorithms (GA) [34] and deep reinforcement learning (RL) [35, 36] which are capable of creating new topologies. Nevertheless, ANNs still excel in the domain of PT as shown in [8] and its use is validated. Moreover, [3] has proved that it is able to provide instant results for circuits once trained properly.

The first and crucial phase of producing a high-performing ANN model is to ensure accurate modelling of analogue circuit performance with quality datasets. Hence, the complete datasets will be generated via OCEAN scripting in Cadence Virtuoso for the selected topology that acts as ground truths for the ANN model. Differing from [3], this project will conduct a preprocessing step on the dataset with the modified g_m / I_d methodology which is a function of g_m / I_d , V_{DS} , and L respectively, as presented in [22]. This innovative method will be able to better approximate submicron devices which are dominated by short channel effects such as the DIBL instead of sweeping random widths, W and lengths, L . This preprocessing step is vital in filtering and eliminating any unwanted values that lead to linear or cutoff regions of transistors. The described steps are illustrated in Figure 3.3 below.

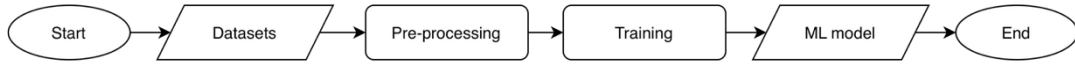


Figure 3.2: Flowchart of training an ANN model

After the model has been trained, it can be used to predict circuit parameters for the selected topology, which will be compared against SPICE simulations for validation. Again, for clarification, the general structure of the ANN model used is shown in Figure 3.4 below.

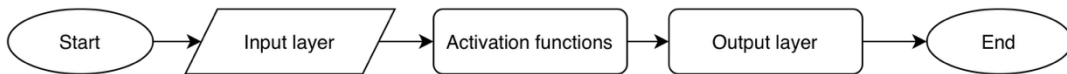


Figure 3.3: Structure of an ANN

Table 2.2 shows some of the most commonly used activation functions in an ANN. However, [2] has showed that ReLU has an advantage over other activation functions, specifically the sigmoid function. Hence, it will be utilised in this project too. Moreover, the Adam optimizer will be employed to update the learning rates in Eq 2.3 adaptively.

3.2 Tools and Platforms

The primary tool used for this project will mainly be the Python coding language, together with its relevant libraries for machine learning, including but not limited to PyTorch and TensorFlow. Besides that, Cadence EDA will be used to generate the initial datasets for the machine learning model which will run in a Red Hat Linux distribution. The deployment platform will be a locally hosted computer or machine running Windows OS.

3.3 Project Management

The Gantt chart in Figure 3.4 shows the time allocation for each stages of the project, starting with the determination of project field. After that, concepts were developed before moving on to a literature review on machine learning. Much focus was put on selecting the appropriate topology and researching the dataset before training a preliminary model for assessment.

FYP 1																
Activities	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	
Topic Selection & Proposal																
Concept Development																
Literature Review (Machine Learning)																
Methodology Design (Flowchart)																
Topology Selection																
Dataset Research																
Preliminary Model Training																
Results Analysis																
Slide Prep & Presentation																
FYP 1 Report Writing																

Figure 3.4: Gantt chart showing project schedule

Next, the Gantt chart in Figure 3.5 shows the planned schedule for the second half of this project: starting off with dataset generation for RAFFC RNMC topology together with the preprocessing steps. Then, the data will be used to train the ANN model, whose results are to be validated against off-the-shelf simulators, i.e. Cadence or LTSpice. Then, the algorithm will be fine-tuned for the best performance. If time

permits, further concepts will be explored and applied. Finally, all relevant achievements will be properly documented for academic purposes.

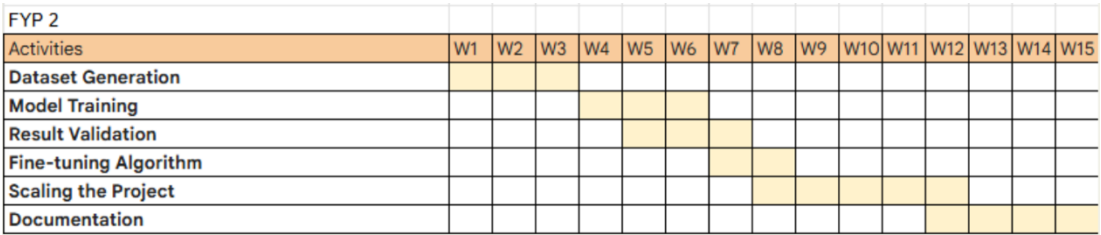


Figure 3.5: Gantt chart for upcoming schedule

As most this project only requires software simulation, no real costs are incurred other than the use of the Cadence License, which is readily provided by the university. All costs are tabulated in Table 3.1 below.

Table 3.1: Cost of project

Tools/Software/Equipment	Price (RM)
Cadence licensing	Provided by campus
Python development language	FOC
Linux distribution	Open source

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Proof of Concept

As a proof-of-concept of the proposed ANN model in predicting transistor sizing, a preliminary performance model has been generated to optimize the design of a **fully differential telescopic cascode two-stage voltage amplifier (TSVA)**. The schematics of the circuit is shown in the figure below.

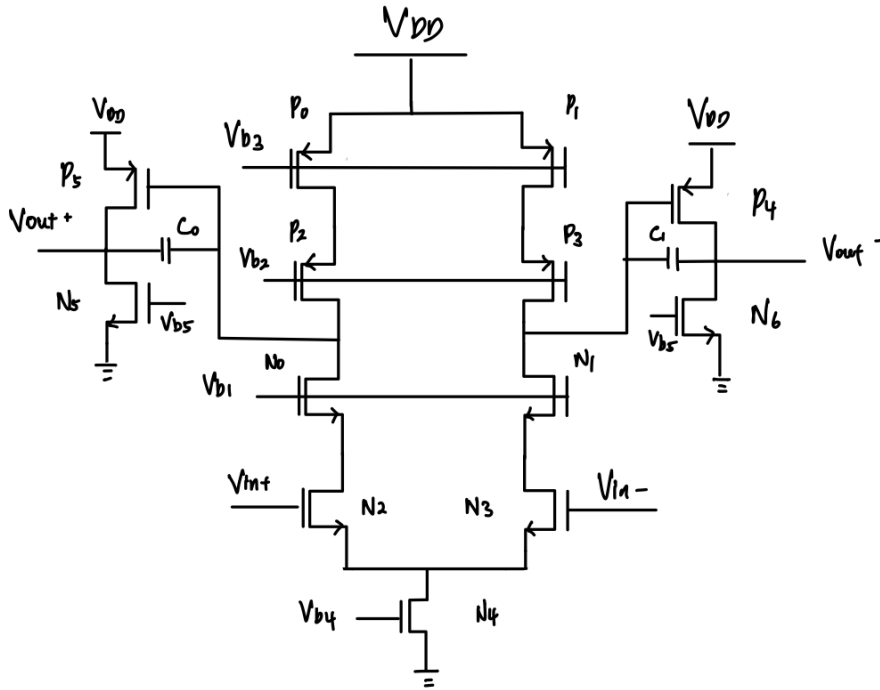


Figure 4.1: Schematics of two-stage voltage amplifier circuit

The first stage consists of a telescopic cascode topology with differential inputs, V_{in+} and V_{in-} . Then, the differential outputs are fed to the second stage which is in a common source amplifier configuration. This specific configuration is more sophisticated than a simple 5-transistor OTA, and requires balancing of cascode branches and common-mode feedback (CMFB), which could test out the ANN's ability in predicting transistor sizing.

Figure 4.2 below shows the netlist of the fully differential telescopic cascode TSVA circuit as shown in Figure 4.1, which is auto-generated by Cadence. Among it contains variables, i.e. WNOUT, WPOUT, WCS, etc. These variables will be injected with different values via OCEAN scripting, for example:

desVar(WNOUT 1.5u)

Which denotes WNOUT as $1.5\mu\text{m}$.

```
// Library name: TSVA
// Cell name: TwoStage
// View name: schematic
N6 (Vout\ - Vb5 0 GND) nmos w=WNOUT l=45.0n as=WNOUT * 2.5 * (45.0n) ad=WNOUT * 2.5 * (45.0n) \
    ps=(2 * WNOUT) + (5 * (45.0n)) pd=(2 * WNOUT) + (5 * (45.0n)) m=5 \
    region=sat
N5 (Vout\ + Vb5 0 GND) nmos w=WNOUT l=45.0n as=WNOUT * 2.5 * (45.0n) ad=WNOUT * 2.5 * (45.0n) \
    ps=(2 * WNOUT) + (5 * (45.0n)) pd=(2 * WNOUT) + (5 * (45.0n)) m=5 \
    region=sat
N4 (net7 Vb4 0 GND) nmos w=WCS l=45.0n as=WCS * 2.5 * (45.0n) ad=WCS * 2.5 * (45.0n) \
    ps=(2 * WCS) + (5 * (45.0n)) pd=(2 * WCS) + (5 * (45.0n)) m=4 \
    region=sat
N3 (net6 Vin\ - net7 GND) nmos w=WN1 l=45.0n as=WN1 * 2.5 * (45.0n) ad=WN1 * 2.5 * (45.0n) \
    ps=(2 * WN1) + (5 * (45.0n)) pd=(2 * WN1) + (5 * (45.0n)) m=4 \
    region=sat
N2 (net5 Vin\ + net7 GND) nmos w=WN1 l=45.0n as=WN1 * 2.5 * (45.0n) ad=WN1 * 2.5 * (45.0n) \
    ps=(2 * WN1) + (5 * (45.0n)) pd=(2 * WN1) + (5 * (45.0n)) m=4 \
    region=sat
N1 (net9 Vb1 net5 GND) nmos w=WN1 l=45.0n as=WN1 * 2.5 * (45.0n) ad=WN1 * 2.5 * (45.0n) \
    ps=(2 * WN1) + (5 * (45.0n)) pd=(2 * WN1) + (5 * (45.0n)) m=4 \
    region=sat
N0 (net8 Vb1 net5 GND) nmos w=WN1 l=45.0n as=WN1 * 2.5 * (45.0n) ad=WN1 * 2.5 * (45.0n) \
    ps=(2 * WN1) + (5 * (45.0n)) pd=(2 * WN1) + (5 * (45.0n)) m=4 \
    region=sat
P5 (Vout\ + net8 VDD VDD) pmos w=WPOUT l=45.0n as=WPOUT * 2.5 * (45.0n) ad=WPOUT * 2.5 * (45.0n) \
    ps=(2 * WPOUT) + (5 * (45.0n)) pd=(2 * WPOUT) + (5 * (45.0n)) m=5 \
    region=sat
P4 (Vout\ - net9 VDD VDD) pmos w=WPOUT l=45.0n as=WPOUT * 2.5 * (45.0n) ad=WPOUT * 2.5 * (45.0n) \
    ps=(2 * WPOUT) + (5 * (45.0n)) pd=(2 * WPOUT) + (5 * (45.0n)) m=5 \
    region=sat
P3 (net9 Vb2 net2 VDD) pmos w=WP1 l=45.0n as=WP1 * 2.5 * (45.0n) ad=WP1 * 2.5 * (45.0n) \
    ps=(2 * WP1) + (5 * (45.0n)) pd=(2 * WP1) + (5 * (45.0n)) m=4 \
    region=sat
P2 (net8 Vb2 net1 VDD) pmos w=WP1 l=45.0n as=WP1 * 2.5 * (45.0n) ad=WP1 * 2.5 * (45.0n) \
    ps=(2 * WP1) + (5 * (45.0n)) pd=(2 * WP1) + (5 * (45.0n)) m=4 \
    region=sat
P1 (net2 Vb3 VDD VDD) pmos w=WP1 l=45.0n as=WP1 * 2.5 * (45.0n) ad=WP1 * 2.5 * (45.0n) \
    ps=(2 * WP1) + (5 * (45.0n)) pd=(2 * WP1) + (5 * (45.0n)) m=4 \
    region=sat
P0 (net1 Vb3 VDD VDD) pmos w=WP1 l=45.0n as=WP1 * 2.5 * (45.0n) ad=WP1 * 2.5 * (45.0n) \
    ps=(2 * WP1) + (5 * (45.0n)) pd=(2 * WP1) + (5 * (45.0n)) m=4 \
    region=sat
C3 (Vout\ - 0) capacitor c=C0 m=1
C2 (Vout\ + 0) capacitor c=C0 m=1
C1 (net9 Vout\ -) capacitor c=C1 m=1
C0 (net8 Vout\ +) capacitor c=C1 m=1
V8 (Vin\ - GND) vsource type=dc dc=Vbmain mag=2m phase=180
V7 (Vin\ + GND) vsource type=dc dc=Vbmain mag=2m phase=0
V6 (Vb5 GND) vsource type=dc dc=Vb5
V5 (Vb4 GND) vsource type=dc dc=Vb4
V4 (Vb3 GND) vsource type=dc dc=Vb3
V3 (Vb2 GND) vsource type=dc dc=Vb2
V2 (Vb1 GND) vsource type=dc dc=Vb1
V1 (VDD GND) vsource type=dc dc=VDD
V0 (GND 0) vsource type=dc dc=0
```

Figure 4.2: Netlist of TSVA generated by Cadence

Figure 4.3 below shows the OCEAN script used in generating the dataset by varying the values of WPOUT, WP1, WNOUT, WN1, WCS and C1 in a nested for loop.

```

; 1. SYSTEM SETUP
simulator( 'spectre )
design( "/home/asal/Shared/Netlists/TSVA/netlist" ) ; Update path if needed
resultsDir( "/home/asal/Shared/Results/TSVA" )
modelFile(
    '{"/home/asal/Shared/Model/45n270AModels.sp" ""}
)

; 2. ANALYSIS SETUP
analysis('ac ?start "10" ?stop "100G" )
analysis('dc ?saveOpptoint t )
envOption( 'analysisOrder list("dc" "ac" ) )
temp( 27 )

; 3. DEFINE FIXED CONSTANTS
; Assumed fixed at nominal values
desVar( "C0" 200f ) ; Load Capacitor (Assumed fixed)
desVar( "Vb1" 1.5 )
desVar( "Vb2" 1.8 )
desVar( "Vb3" 2.7 )
desVar( "Vb4" 620m )
desVar( "Vb5" 1 )
desVar( "Vbmain" 740m )
desVar( "VDD" 3.3 )

; 4. DEFINE SWEEP PARAMETERS (Extracted from TSVA.csv)
c1_vals = '(1.5e-13 2e-13 2.5e-13)
wcs_vals = '(4e-06 4.5e-06 5e-06 5.5e-06 6e-06)
wn1_vals = '(2.5e-06 2.75e-06 3e-06 3.25e-06 3.5e-06 3.75e-06 4e-06 4.25e-06 4.5e-06)
wnout_vals = '(1.5e-06 2.5e-06 3e-06 3.5e-06 4.5e-06)
wp1_vals = '(2.5e-06 2.75e-06 3e-06 3.25e-06 3.5e-06 3.75e-06 4e-06 4.25e-06 4.5e-06)
wpout_vals = '(1.5e-06 2.5e-06 3e-06 3.5e-06 4.5e-06)

; 5. INITIALIZE OUTPUT FILE
out_file = ".TSVA_reproduction.csv"
filePtr = outfile(out_file "w")
; Write Header matching your CSV format
fprintf(filePtr "C1,WNOUT,WPOUT,WCS,WN1,WP1,Bandwidth,PowerConsumption,VoltageGain\n")

; 6. NESTED SWEEP LOOPS (6 Levels)
foreach( c1 c1_vals
    desVar( "C1" c1 )

    foreach( wcs wcs_vals
        desVar( "WCS" wcs )

        foreach( wn1 wn1_vals
            desVar( "WN1" wn1 )

            foreach( wnout wnout_vals
                desVar( "WNOUT" wnout )

                foreach( wp1 wp1_vals
                    desVar( "WP1" wp1 )

                    foreach( wpout wpout_vals
                        desVar( "WPOUT" wpout )

                        ; --- RUN SIMULATION ---
                        run()

                        ; --- EXTRACT MEASUREMENTS ---
                        ; Calculate Voltage Gain (dB)
                        v_gain = ymax(dB20(((VF("/Vout+") - VF("/Vout-")) / (VF("/Vin+") - VF("/Vin-")))))

                        ; Calculate Bandwidth (Hz) - 3dB drop from DC max
                        bw = cross(dB20(((VF("/Vout+") - VF("/Vout-")) / (VF("/Vin+") - VF("/Vin-")))))
                            (ymax(dB20(((VF("/Vout+") - VF("/Vout-")) / (VF("/Vin+") - VF("/Vin-"))))) - 3) 1 "falling" nil nil nil)

                        ; Calculate Power (Watts)
                        pwr = getData(":pwr" ?result "dcOp")

                        ; --- WRITE TO FILE ---
                        ; Check if simulation converged (Bandwidth is not nil)
                        if( bw != nil then
                            fprintf(filePtr "%g,%g,%g,%g,%g,%g,%g,%g,%g\n"
                                c1 wnout wpout wcs wn1 wp1 bw pwr v_gain
                            )
                        else
                            ; Handle failed points (optional)
                            fprintf(filePtr "%g,%g,%g,%g,%g,%g,%g,%g,%g\n"
                                c1 wnout wpout wcs wn1 wp1 pwr v_gain
                            )
                        )
                    )
                )
            )
        )
    ) ; End WPOUT loop
) ; End WP1 loop
) ; End WNOUT loop
) ; End WN1 loop
) ; End WCS loop
) ; End C1 loop

; 7. FINISH
close(filePtr)
printf("Completed sweep. Results saved to %s\n" out_file)

```

Figure 4.3: OCEAN script used in generating data

Figure 4.4 below shows part of the dataset generated by OCEAN scripting on the netlist as shown in Figure 4.1. With the script in Figure 4.3, the different values of C1, WPOUT, WNOUT, WCS WN1 and WP1 are injected into the netlist to produce different values of bandwidth, power consumption and voltage gain.

All the data will be used to train the proposed ANN model. However, as can be seen in row 6 of Figure 4.4, the value of $WP1 = 2.5e-06$ has led to a voltage gain of only 1.25 because the PMOS is operating in the linear region. This illustrates the importance of **pre-processing datasets** to prevent unnecessary garbage data when training the model. This will be done in the near future when conducting FYP2.

```

1  C1,WNOUT,WPOUT,WCS,WN1,WP1,Bandwidth,PowerConsumption,VoltageGain
2  2e-13,1.5e-06,1.5e-06,6e-06,3e-06,3e-06,12300000.0,0.0366,33.41
3  2e-13,1.5e-06,2.5e-06,6e-06,3e-06,3e-06,13400000.0,0.0586,28.54
4  2e-13,1.5e-06,3.5e-06,6e-06,3e-06,3e-06,13300000.0,0.0801,27.07
5  2e-13,1.5e-06,4.5e-06,6e-06,3e-06,3e-06,13100000.0,0.101,26.3
6  2e-13,1.5e-06,1.5e-06,6e-06,3e-06,2.5e-06,11900000.0,0.0574,1.25
7  2e-13,1.5e-06,1.5e-06,6e-06,3e-06,2.75e-06,45700000.0,0.0546,11.7
8  2e-13,1.5e-06,2.5e-06,6e-06,3e-06,2.75e-06,48300000.0,0.0843,8.62
9  2e-13,1.5e-06,3.5e-06,6e-06,3e-06,2.75e-06,49200000.0,0.112,7.34
10 2e-13,1.5e-06,4.5e-06,6e-06,3e-06,2.75e-06,49400000.0,0.138,6.6
11 2e-13,1.5e-06,1.5e-06,6e-06,3e-06,3.25e-06,24900000.0,0.0217,31.53
12 2e-13,1.5e-06,2.5e-06,6e-06,3e-06,3.25e-06,16700000.0,0.0231,38.22
13 2e-13,1.5e-06,3.5e-06,6e-06,3e-06,3.25e-06,12500000.0,0.0245,41.92
14 2e-13,1.5e-06,4.5e-06,6e-06,3e-06,3.25e-06,10800000.0,0.0265,43.61
15 2e-13,1.5e-06,1.5e-06,6e-06,3e-06,3.5e-06,80600000.0,0.0213,20.54
16 2e-13,1.5e-06,2.5e-06,6e-06,3e-06,3.5e-06,60900000.0,0.0221,25.78
17 2e-13,1.5e-06,3.5e-06,6e-06,3e-06,3.5e-06,47400000.0,0.0229,29.48
18 2e-13,1.5e-06,4.5e-06,6e-06,3e-06,3.5e-06,39200000.0,0.0236,31.91
19 2e-13,1.5e-06,1.5e-06,6e-06,3e-06,3.75e-06,10900000.0,0.0211,18.28
20 2e-13,1.5e-06,2.5e-06,6e-06,3e-06,3.75e-06,88400000.0,0.0218,22.16
21 2e-13,1.5e-06,3.5e-06,6e-06,3e-06,3.75e-06,72400000.0,0.0224,25.34
22 2e-13,1.5e-06,4.5e-06,6e-06,3e-06,3.75e-06,60300000.0,0.0229,27.81
23 2e-13,1.5e-06,1.5e-06,6e-06,3e-06,4e-06,12700000.0,0.021,17.52
24 2e-13,1.5e-06,2.5e-06,6e-06,3e-06,4e-06,10900000.0,0.0216,20.41
25 2e-13,1.5e-06,3.5e-06,6e-06,3e-06,4e-06,92800000.0,0.0221,22.94

```

Figure 4.4: Dataset generated by OCEAN scripting

By applying the innovative modified g_m/I_d methodology, the size of the dataset can be significantly reduced. Currently, there are around 19000 data produced by the OCEAN script, causing long training time. This is an area of improvement left for FYP2.

Figure 4.5 shows the code snippet for the preliminary MLP ANN, which consists of seven hidden layers made up of linear and Rectified Linear Unit (ReLU) activation functions, that maps design specifications to circuit parameters (inverse mapping).

```
class Model500RELU(nn.Module):
    def __init__(self, input_count=2, output_count=2):
        super(Model500RELU, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_count, 200),
            nn.ReLU(),
            nn.Linear(200, 300),
            nn.ReLU(),
            nn.Linear(300, 500),
            nn.ReLU(),
            nn.Linear(500, 500),
            nn.ReLU(),
            nn.Linear(500, 300),
            nn.ReLU(),
            nn.Linear(300, 200),
            nn.ReLU(),
            nn.Linear(200, output_count)
        )

    def forward(self, x):
        return self.network(x)
```

Figure 4.5: Code snippet for ANN model

The fully connected layers in the ANN structure are represented as:

$$Input \rightarrow 200 \rightarrow 300 \rightarrow 500 \rightarrow 500 \rightarrow 300 \rightarrow 200 \rightarrow Output$$

This “diamond” shape is a common architecture designed to capture high-dimensional features in the middle layers before compressing parameters to specific output metrics [2]. Nonetheless, the number of layers and nodes are subject to change to achieve the best performance.

Also, the input and output parameters of the model are also shown in a tabular format as shown below, corresponding to the dataset generated by OCEAN scripting in Figure 4.4:

Table 4.1: Table showing inputs and outputs of ANN model

Input	Output
Bandwidth	WN1
	WP1
Power consumption	WCS
	WNOUT
Voltage gain	WPOUT
	C0
	C1

4.2 Results and Discussion

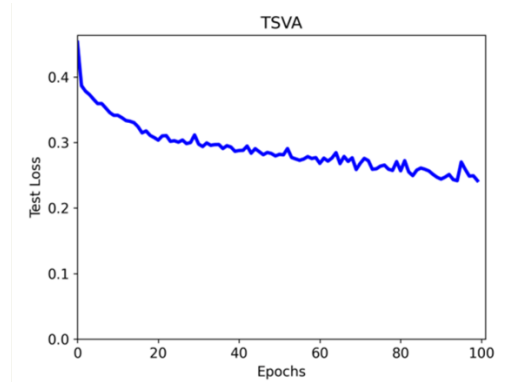


Figure 4.6: Test loss of preliminary ANN model

The figure above shows the test loss, which indicates the error between the model's prediction and the known data studied by the model. The final test loss is 0.2322 for the model trained on 100 epochs.

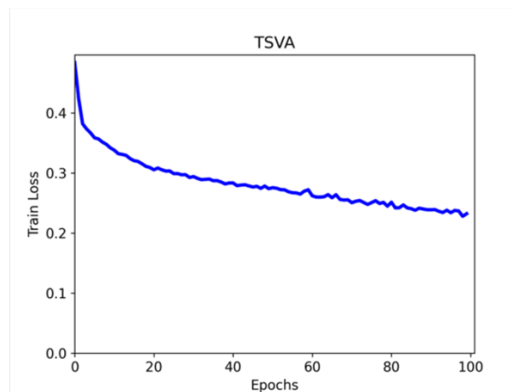


Figure 4.7: Training loss of preliminary ANN model

The figure above shows the training loss, which indicates the error between the model's prediction and the previously unknown data studied by the model. The final training loss is 0.2418 for the model trained on 100 epochs.

From these results, the error is still significant. However, it can be seen that the errors are still steadily decreasing, meaning that it can be further reduced by increasing the number of epochs. Another possible strategy is to split the dataset to:

Training (70%) : Validation (20%) : Testing (10%)

Implementing these strategies can further increase the accuracy of the performance model.

Furthermore, training time for the ANN model is shown in Figure 4.8, taking only 60.78 seconds, which is relatively fast, compared to [23]. Of course, this is only for the measly 100 epochs, where the time taken is expected to increase exponentially when number of epochs is scaled.

```
Total time for 100 epochs: 60.78 seconds
```

Figure 4.8: Terminal output showing elapsed time for ANN training

4.3 Chapter Summary

In this chapter, a preliminary MLP ANN model was built to predict the transistor sizing on a differential telescopic cascode two-stage voltage amplifier (TSVA). Datasets were generated via OCEAN scripting which were then used to train the ANN model. The ANN model consisted of 7 hidden layers and were trained with 100 epochs, producing test loss of 0.2322 and a validation loss of 0.2418, taking a total training time of 60.78 seconds. Finally, multiple strategies will be attempted to decrease these errors in FYP2.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusions

In conclusion, this project has successfully implemented a preliminary MLP ANN framework for automating the sizing of analogue circuit, i.e. TSVA, thus achieving the primary objective of creating an EDA. As a proof-of-concept, the framework was validated on the fully differential telescopic cascode TSVA proving that the proposed idea is feasible. Comprehensive dataset was generated with OCEAN scripting in Cadence by sweeping key design values (W, L and C) to model the circuit's non-linear behavior. The 7-layer MLP achieved convergence with a test loss of roughly 0.23, indicating that the network successfully learned the fundamental relationships between transistor sizing and performance (gain, power and bandwidth).

5.2 Contributions to Knowledge

This work contributes to the field of Electronic Design Automation (EDA) by reducing the iterations in design processes. By accurately modelling non-linear trade-offs in analogue circuits, ANNs are able to replace thousands of slow SPICE simulations, providing instant results. This greatly improves the time-to-market of analogue IP blocks. Moreover, the existence of such EDA tool also benefits beginners to develop quality analogue circuits, effectively lowering the level of entry. Hopefully, it will lead to more complex and efficient topologies to be developed.

5.3 Future Works

In the future, the scope will transition to the RNMC topology which is much more complex compared to the TSVA. This will test the ANN's ability in handling higher-dimension design spaces, proving its suitability as the best inverse mapper for non-linear problems. Furthermore, advanced data preprocessing will be done on the generated dataset with the modified g_m/I_d methodology to remove "garbage data". All of these aim to increase the accuracy of the ANN.

REFERENCES

1. Mina, R., C. Jabbour, and G.E. Sakr, *A Review of Machine Learning Techniques in Analog Integrated Circuit Design Automation*. Electronics, 2022. **11**(3).
2. Lourenço, N., et al., *On the Exploration of Promising Analog IC Designs via Artificial Neural Networks*. 2022.
3. Devi, S., G. Tilwankar, and R. Zele, *Automated Design of Analog Circuits using Machine Learning Techniques*, in *2021 25th International Symposium on VLSI Design and Test (VDATE)*. 2021. p. 1-6.
4. Dutta, R., et al., *Automated Deep Learning Platform For Accelerated Analog Circuit Design* 2024.
5. Chai, Z., et al., *CircuitNet: an open-source dataset for machine learning applications in electronic design automation (EDA)*. Science China Information Sciences, 2022. **65**(12).
6. Chen, Y.Y. and S.Y. Fang, *Optimizing Analog Circuit Design Through a Machine Learning - Assisted Evolutionary Algorithm*. Electronics Letters, 2025. **61**(1).
7. Noori Zadeh, D. and M.B. Elamien, *Generative AI for Analog Integrated Circuit Design: Methodologies and Applications*. IEEE Access, 2025. **13**: p. 58043-58059.
8. Chen, Z., et al., *MACRO: Multi-agent Reinforcement Learning-based Cross-layer Optimization of Operational Amplifier*, in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. 2024. p. 423-428.
9. Budak, A.F., et al., *An Efficient Analog Circuit Sizing Method Based on Machine Learning Assisted Global Optimization*. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022. **41**(5): p. 1209-1221.
10. Grasso, A.D., G. Palumbo, and S. Pennisi, *Advances in Reversed Nested Miller Compensation*. IEEE Transactions on Circuits and Systems I: Regular Papers, 2007. **54**(7): p. 1459-1470.
11. Mita, R., G. Palumbo, and S. Pennisi, *Design guidelines for reversed nested miller compensation in three-stage amplifiers*. IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing, 2003. **50**(5): p. 227-233.
12. Zhu, F., et al., *Feedforward Reversed Nested Miller Compensation Techniques for Three-Stage Amplifiers*.
13. Grasso, A.D., G. Palumbo, and S. Pennisi, *Active Reversed Nested Miller Compensation for Three-Stage Amplifiers*.
14. Kin-Pui, H., et al., *Reversed nested miller compensation with voltage buffer and nulling resistor*. IEEE Journal of Solid-State Circuits, 2003. **38**(10): p. 1735-1738.
15. Li, Y., et al., *Exploring a Machine Learning Approach to Performance Driven Analog IC Placement*. 2022.
16. Jafari, A., S. Sadri, and M. Zekri, *Design Optimization of Analog Integrated Circuits by Using Artificial Neural Networks*. 2010.

17. Kufel, J., et al., *What Is Machine Learning, Artificial Neural Networks and Deep Learning?-Examples of Practical Applications in Medicine*. Diagnostics (Basel), 2023. **13**(15).
18. Grossia, E. and M. Buscemab, *Introduction to artificial neural networks*. 2012.
19. Poonam, M. Duhan, and H. Saini, *Design of Two Stage Op-Amp*. 2013.
20. Parihar, R.S. and A. Gupta, *Design of a Fully Differential Two-Stage CMOS Op-Amp for High Gain, High Bandwidth Applications*.
21. Pinjare, S.L., et al., *A Gm/Id Based Methodology for Designing Common Source Amplifier*, in *2018 2nd International Conference on Micro-Electronics and Telecommunication Engineering (ICMETE)*. 2018. p. 304-307.
22. Pollissard-Quatremère, G., G. Gosset, and D. Flandre, *A modified gm/ID design methodology for deeply scaled CMOS technologies*. Analog Integrated Circuits and Signal Processing, 2013. **78**(3): p. 771-784.
23. Zhao, Z. and L. Zhang, *Analog Integrated Circuit Topology Synthesis With Deep Reinforcement Learning*. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2022. **41**(12): p. 5138-5151.
24. Yelten, M.B., et al., *Demystifying Surrogate Modeling for Circuits and Systems*. IEEE Circuits and Systems Magazine, 2012. **12**(1): p. 45-63.
25. Dutta, R., *Learning of Multi-Dimensional Analog Circuits Through Generative Adversarial Network (GAN)*. 2019.
26. Lin, H. and P. Li, *Circuit Performance Classification With Active Learning Guided Sampling for Support Vector Machines*. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2015. **34**(9): p. 1467-1480.
27. Wang, F. and X. Li, *Correlated Bayesian Model Fusion*, in *Proceedings of the 53rd Annual Design Automation Conference*. 2016. p. 1-6.
28. Hershenson, M.d.M., S.P. Boyd, and T.H. Lee, *Optimal Design of a CMOS Op-Amp via Geometric Programming*. 2001.
29. Wang, Y., M. Orshansky, and C. Caramanis, *Enabling Efficient Analog Synthesis by Coupling Sparse Regression and Polynomial Optimization*. 2014.
30. Hoang, T., et al., *Novel Methods for Improved Particle Swarm Optimization in Designing the Bandgap Reference Circuit*. IEEE Access, 2023. **11**: p. 139964-139978.
31. Lyu, W., et al., *An Efficient Bayesian Optimization Approach for Automated Optimization of Analog Circuits*. IEEE Transactions on Circuits and Systems I: Regular Papers, 2018. **65**(6): p. 1954-1967.
32. Lyu, W., et al., *Multi-objective bayesian optimization for analog/RF circuit synthesis*, in *Proceedings of the 55th Annual Design Automation Conference*. 2018. p. 1-6.
33. Liou, G.-H., et al., *Classifying Analog and Digital Circuits with Machine Learning Techniques toward Mixed-Signal Design Automation*. 2018.
34. Cohen, M.W., M. Aga, and T. Weinberg, *Genetic Algorithm Software System for Analog Circuit Design*. Procedia CIRP, 2015. **36**: p. 17-22.
35. Settaluri, K., et al., *AutoCKT: Deep Reinforcement Learning of Analog Circuit Design*. 2023.
36. Morales, E.F. and J.H. Zaragoza, *An Introduction to Reinforcement Learning*.